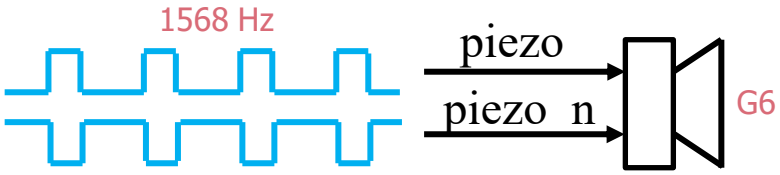
Exercise 19: piezo_drv

Piezo Bender

- Certain materials (e.g., quartz crystal) produce voltage when they are mechanically stressed, and their shape deform when voltage is applied → piezoelectric effect

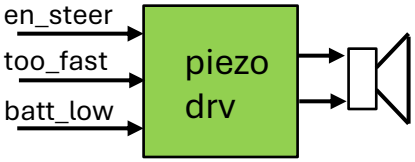
- Piezo bender – a "speaker" that uses piezoelectric effect to generate sound
  - A metal layer sandwiched between 2 piezoelectric ceramic layers
  - When voltage applied, one contracts and one expands

- We can drive the piezo bender using GPIO pins on the FPGA
  - Use square wave of the frequency we want to generate a tone for
    - Piezo will respond to signals in the 300Hz to 7kHz range
  - Duty cycle is not so important (anything from 20% to 80% will do)
  - We will drive differentially for increased amplitude



Exercise 19: piezo_drv

- The Piezo element is used to provide warnings.
  - Warning to people in vicinity when rider on
  - Warn rider when going too fast
  - Warn rider when battery is getting low



- 3 Signals come to piezo to inform of various conditions.

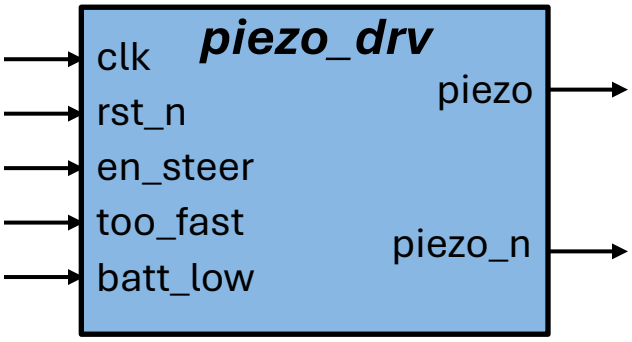
- During normal operation *(normal rider on and riding case),* **en_steer** will be high, and you should play the tune of *"charge!" fanfare* once every 3 seconds



- If you are going **too_fast** on the Segway, you should play the first 3 notes of *"charge!" fanfare* (G6, C7, E7) continuously. **too_fast** tone has priority over all others as it is indicating a dangerous condition.
- If **batt_low** is asserted *(and it is not **too_fast**)* then *"charge!" fanfare* should be played backwards once every 3 seconds, regardless the state of **en_steer**.
- If no input signals are asserted the piezo should be silent.

Exercise 19: piezo_drv

- Your interface should looks like this:



Signal	Dir	Description
clk,rst_n	In	50MHz clk & asynch reset
en_steer	In	"normal" operation
too_fast	In	Priority over other inputs
batt_low	In	Charge backwards
piezo,piezo_n	out	Differential piezo drive

- You are to design the FSM and associated datapath needed
- Since this module uses real-time values, it is good to have a **fast_sim** parameter to speed up simulation of this block
  - Shorten 3s duration
  - Shorten note durations
  - Speed up frequency of notes?

Work as 2 groups of 2 within your project team
→ This way your team will have 2 possible solutions to pick from

Exercise 19: piezo_drv

- "Charge!" Fanfare as frequencies & **simplified** durations:

Note	Freq	Duration
G6 (G in octave 6)	1568	2^{23} clocks
C7 (C in octave 7)	2093	2^{23} clocks
E7	2637	2^{23} clocks
G7	3136	$2^{23} + 2^{22}$ clocks
E7	2637	2^{22} clocks
G7	3136	2^{24} clocks



NOTE: The too_fast sound *(G6,C7,E7 played for 2^{23} clocks each)* is comprised of 3-notes that were already part of "Charge!"

- Once "Charge!" is started, it is allowed to complete (no need to preempt "charge!" if battery becomes low)

- You will probably need 3 counters in the design:
  - A note duration timer – how long a note is being played
  - A note frequency/period timer – establishes what note is being played
  - A repeat timer – times out every 3s to repeat tune

Exercise 19: piezo_drv

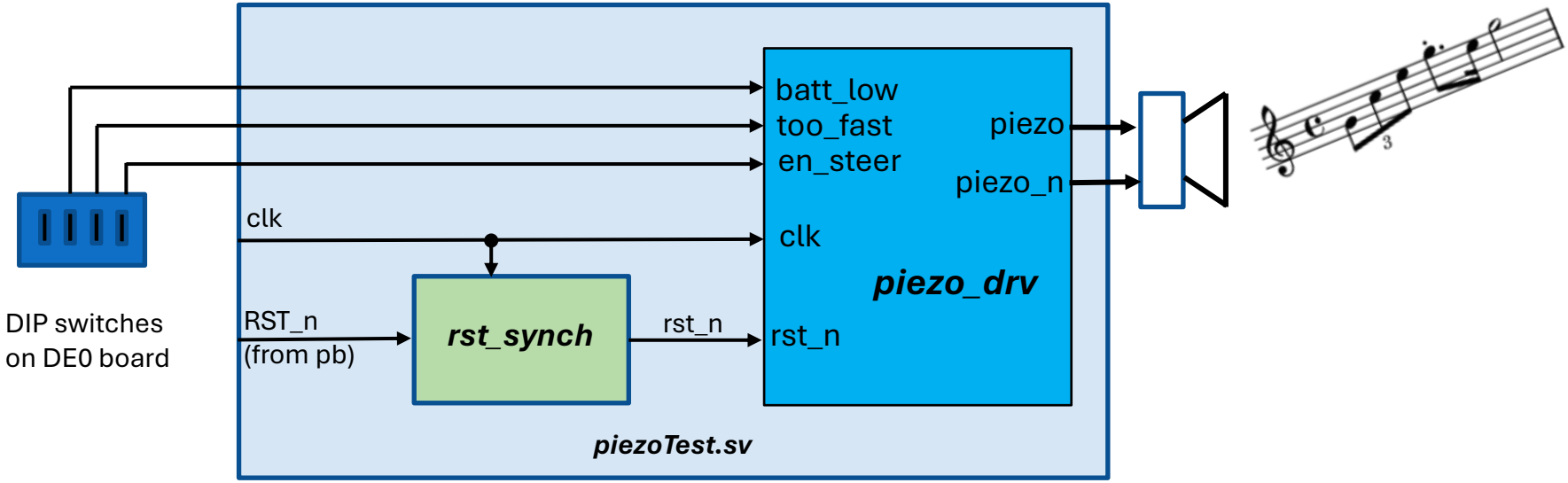
- The duration of the notes (2^{23} clock cycles) is quite long for simulation *(especially when we get to fullchip simulations where **piezo_drv** will be one of many blocks being simulated)*
  - We will use **fast_sim** parameter to speed up simulation, and default it to true

- When FAST_SIM=1
  - 3s should be 1/64 of normal
  - duration of the notes should be 1/64 their normal length
  - all frequencies should be 64X their normal
- **HINT**: Use a generate if statement to create an amount by which you increment your counter (i.e., increment by 1 or by 64 depending on fast_sim)

- Create a simple testbench (**piezo_drv_tb.sv**) that simply instantiates DUT, applies clk and reset, and asserts the inputs in order.
  - **Piezo** and **piezo_n** should not be toggling before any of the inputs are asserted
  - Once your testbench is passing (visual inspection of piezo/piezo_n and state transition is enough), move on to testing with the DE0-nano

- **Submit:** piezo_drv.sv, piezo_drv_tb.sv

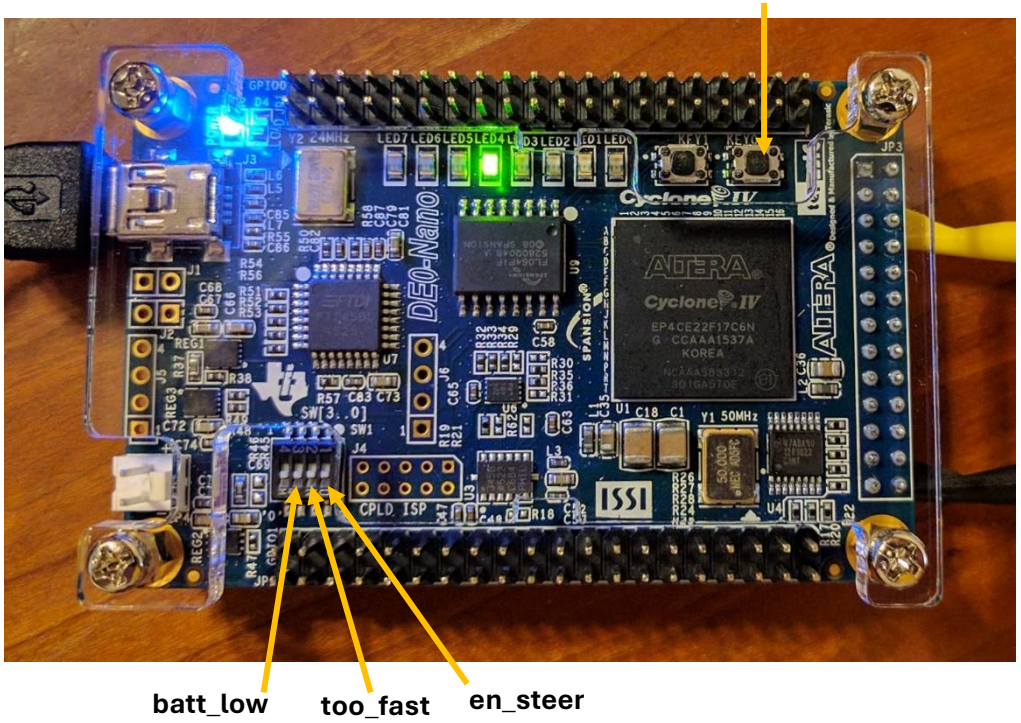
Exercise 19: piezo_drv



- Download the provided testing harness **piezoTest.sv,** and Quartus project and settings file (**piezoTest.qpf, piezoTest.qsf**)
- Copy **reset_synch.sv** from your previous exercise
- You are now ready to map your **piezo_drv.sv** to the DE0 and test in “real life”. Don’t forget to set FAST_SIM=0 since we are testing it on real HW now.
- Ensure that project builds with no errors

Demo

- Call anyone on the teaching staff over to demo on a DE0 test platform
- Bonus credit: +1% if your group (of 2) demos a correct implementation by Friday



Pin mapping shown below:

